

Rogert

Osoba 2 - Prymitywy 3D, przeszkody i kolizje

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/PrimitiveNode.h, Engine/PrimitiveNode.cpp, Demo/Obstacles.cpp

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Rogert).

```
pgk/
|
|-- Engine/
|   |-- Math3D.* SceneNode.*           (Lukasz)
|   |-- PrimitiveNode.h <<< MOJE      (wszystkie kształty 3D)
|   |-- PrimitiveNode.cpp <<< MOJE
|   |-- Camera.* Light.* Texture.*    (Kacper)
|   |-- Engine.*                      (Jakub)
|
|-- Demo/
|   |-- Room.cpp                      (Lukasz)
|   |-- Obstacles.cpp <<< MOJE        (kolumny, stożki, skrzynie + kolizje)
|   |-- Targets.cpp                  (Kacper)
|   |-- Gameplay.cpp Constants.h ShootingGallery.h (Jakub)
|
|-- main.cpp CMakeLists.txt          (Jakub)
```

W skrócie: ja robię **kształty** (szescian, walec, sfera, stożek, torus, płaszczyzna) - każdy rysowany własnym kodem OpenGL - oraz **przeszkody** w grze wraz z **kolizjami** (gracz nie przechodzi przez ściany, strzał nie przelatuje przez kolumny).

1. O co chodzi w moich plikach (po ludzku)

Komputer nie wie, czym jest 'szescian'. Trzeba mu podać każdy **wierzcholek** (rog), **normalna** (kierunek, w który patrzy powierzchnia - potrzebny do światła) i **współrzędne UV** (gdzie na teksturze jest dany punkt). To właśnie robię w **PrimitiveNode** - ręcznie, przez glBegin/glEnd. Nie używam gotowców typu glutSolidCube, bo wymaganiem projektu było napisanie geometrii samemu.

W **Obstacles.cpp** używam tych kształtów, żeby postawić w pokoju kolumny, stożki i skrzynie. Dopusuje też **matematykę kolizji**: żeby gracz nie wchodził w przeszkody i żeby przeszkoda zasłaniała strzał.

2. PrimitiveNode - wspólna baza wszystkich kształtów

PrimitiveNode dziedziczy po **SceneNode** (Lukasz) i dodaje **material** (kolory dla światła) oraz **teksturę**. Ma jedną wspólną metodę **renderSelf**, która ustawia material/teksturę, a potem wola **drawGeometry()** - i to właśnie ta funkcja jest różna dla każdego kształtu.

PrimitiveNode.cpp - wspólny render, geometria w drawGeometry()

```
void PrimitiveNode::renderSelf(const Mat4& worldMatrix) const {
    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, ambient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, diffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, specular);
    glMaterialf (GL_FRONT_AND_BACK, GL_SHININESS, shininess);
    glColor3f(diffuse.x, diffuse.y, diffuse.z); // fallback gdy światło off

    bool hasTexture = (surfaceTexture && surfaceTexture->isLoaded());
    if (hasTexture) { glEnable(GL_TEXTURE_2D); surfaceTexture->bind(); }

    glPushMatrix();
    glMultMatrixf(worldMatrix.data()); // ustaw transformację obiektu
    drawGeometry(); // <- nadpisane w każdej podklasie
    glPopMatrix();

    if (hasTexture) { surfaceTexture->unbind(); glDisable(GL_TEXTURE_2D); }
}
```

drawGeometry() jest **czysto wirtualna** (= 0) - to znaczy, że **PrimitiveNode** sam jej nie ma, a każda podklasa MUSI ją napisać. To wzorzec 'szablon metody': baza robi powtarzalne rzeczy (material, transformacja), a detale (jaki kształt) zostawia potomkom.

3. Jak rysuje konkretne kształty

3.1. Sfera - siatka równoleżników i południków

Sfere dzieli na poziome pasy (stacks) i pionowe wycinki (slices), jak globus. Każdy punkt liczy ze współrzędnych sferycznych. Ważne: dla sfery jednostkowej **normalna = pozycja punktu** (kierunek od środka).

SphereNode::drawGeometry() - jeden wierzchołek

```
float nx = sin(theta)*cos(phi);
float ny = cos(theta);
float nz = sin(theta)*sin(phi);
glNormal3f(nx, ny, nz); // normalna = kierunek na zewnątrz
glTexCoord2f(u, v); // u = phi/2pi, v = theta/pi
glVertex3f(nx*radius, ny*radius, nz*radius);
```

3.2. Stożek - dlaczego osobne trojkąty, a nie wachlarz

Stozek mozna narysowac jako TRIANGLE_FAN z jednym wierzchołkiem na czubku. Ale wtedy czubek ma **jedna** normalna, przez co cieniowanie wyglada plasko/brzydko. Dlatego rysuje N osobnych trojkatow - kazdy ma na czubku **usredniona** normalna z dwoch sasiednich scianek. Wyglada duzo lepiej.

ConeNode::drawGeometry() - nachylenie normalnej zalezy od proporcji r:h

```
float L = sqrt(r*r + h*h);
float nh = h / L;    // pozioma skladowa normalnej
float nv = r / L;    // pionowa skladowa (stozek zwęża sie ku gorze)
// normalna sciany bocznej w kierunku kata 'ang':
glNormal3f(cos(ang)*nh, nv, sin(ang)*nh);
```

3.3. Plaszczyzna i skala UV

PlaneNode to zwykly prostokat w plaszczyznie XZ z normalna w gore. Dodalem **setUVScale**, bo na duzej scianie tekstura powtarzala sie zbyt gesto.

PlaneNode::drawGeometry() - UV mnozone przez skale

```
// uMax/vMax decyduja ile razy tekstura sie powtorzy
const float uMax = planeWidth * planeUVScale;
const float vMax = planeDepth * planeUVScale;
glTexCoord2f(0.0f, 0.0f); glVertex3f(-hw, 0, hd);
glTexCoord2f(uMax, vMax); glVertex3f( hw, 0,-hd); // itd.
```

4. Demo/Obstacles.cpp - przeszkody i kolizje

Tu stawiam w pokoju: 6 kolumn (walce), 3 stożki i 2 skrzynie (szesciany). Każda przeszkoda zapisuje w dwóch miejscach: jako **wezel sceny** (zeby ja narysowac) i jako **dane kolizji** (zeby liczyć fizyke).

4.1. Trzy rodzaje kolizji gracza

Gracz ma 'hitbox' - kołko o promieniu PLAYER_RADIUS. Po ruchu sprawdzam, czy wszedł w przeszkode, i jak tak - **wypycham** go na zewnątrz.

Obstacles.cpp - wypychanie gracza z kolumny

```
// kolizja z walcem (kolizja kolowa w płaszczyźnie XZ)
float dx = playerEye_.x - o.base.x;
float dz = playerEye_.z - o.base.z;
float minR = o.radius + PLAYER_RADIUS;
float distSq = dx*dx + dz*dz;
if (distSq < minR*minR && distSq > 0.0001f) {
    float dist = sqrt(distSq);
    float push = (minR - dist) / dist; // ile brakuje do wyjścia
    playerEye_.x += dx * push;         // wypchnij wzdłuż osi środek->gracz
    playerEye_.z += dz * push;
}
```

Dla **skrzyni** (prostokąt) szukam najbliższego punktu na jej brzegu i wypycham gracza o promień. Dla **stożka** traktuje go jak wałec o promieniu podstawy (gracz i tak nie wejdzie pod nawis).

4.2. Przeszkoda zasłania strzał

Gdy gracz strzela, sprawdzam czy **najpierw** nie trafi w przeszkode. Używam funkcji raycast od Lukasa.

Obstacles.cpp - czy przeszkoda jest bliżej niż cel

```
bool ShootingGallery::obstacleBlocksRay(const Vec3& origin,
                                         const Vec3& dir, float maxT) const {
    float t;
    for (const auto& o : cylinderObs_)
        if (rayCylinderIntersect(origin, dir, o.base, o.radius, o.height, t)
            && t > 0.0001f && t < maxT) return true;
    for (const auto& o : coneObs_)
        if (rayConeIntersect(origin, dir, o.base, o.radius, o.height, t)
            && t > 0.0001f && t < maxT) return true;
    for (const auto& o : boxObs_)
        if (rayAABBIntersect(origin, dir, o.boxMin, o.boxMax, t)
            && t > 0.0001f && t < maxT) return true;
    return false;
}
```

maxT to odległość do trafionego celu. Jeżeli któraś przeszkoda jest bliżej ($t < \text{maxT}$), strzał jest zablokowany i liczy się jako pudło - cel był schowany za kolumną.

5. Pytania od wykładowcy - przygotowanie

Przy każdym pytaniu jest pasek **Gdzie** - mówi w którym pliku i w której funkcji szukać odpowiedzi, żebyś w trakcie obrony szybko otworzył właściwe miejsce.

PrimitiveNode - architektura

P: Dlaczego nie używacie gotowych funkcji jak `glutSolidSphere`?

O: Bo wymaganiem projektu było, żeby grupa sama napisała geometrie prymitywów. FreeGLUT ma tylko pomagać z oknem i wejściem. Dlatego każdy kształt rysuje ręcznie przez `glBegin/glEnd`, samodzielnie licząc wierzchołki, normalne i UV.

Gdzie: `PrimitiveNode.cpp` -> `drawGeometry()` w każdej klasie

P: Dlaczego `drawGeometry` jest czysto wirtualna (= 0)?

O: Bo `PrimitiveNode` to klasa abstrakcyjna - nie istnieje 'ogólny prymityw', istnieje konkretny sześcian czy sfera. `=0` wymusza, że każda podklasa musi napisać własne rysowanie. Wspólne rzeczy (material, transformacja, tekstura) są w `renderSelf` w bazie - to wzorzec szablonu metody (template method).

Gdzie: `PrimitiveNode.h` -> `virtual void drawGeometry() const = 0;`

P: Co dokładnie robi `renderSelf` w `PrimitiveNode`?

O: Ustawia material przez `glMaterialfv` (ambient/diffuse/specular/shininess), ustawia kolor fallback przez `glColor3f` (na wypadek wylaczonego światła), włącza tekstury jeżeli jest, robi `glPushMatrix + glMultMatrixf(world) + drawGeometry() + glPopMatrix`, na końcu wylacza tekstury. To powtarzalna otoczka wokół rysowania konkretnego kształtu.

Gdzie: `PrimitiveNode.cpp` -> `PrimitiveNode::renderSelf()`

P: Po co `glPushMatrix` / `glPopMatrix` wokół rysowania?

O: `glPushMatrix` zapisuje aktualną macierz na stosie, `glMultMatrixf` nakłada transformację obiektu, rysujemy, a `glPopMatrix` przywraca poprzedni stan. Dzięki temu transformacja jednego obiektu nie wpływa na następny rysowany.

Gdzie: `PrimitiveNode.cpp` -> `PrimitiveNode::renderSelf()`

P: Czemu ustawiasz i `glMaterial`, i `glColor3f` naraz?

O: `glMaterial` działa gdy oświetlenie jest WŁACZONE. `glColor3f` jest fallbackiem - gdy gracz wylaczy światło (klawisz L), liczy się kolor z `glColor`. Ustawiam oba, żeby obiekt był widoczny w obu trybach.

Gdzie: `PrimitiveNode.cpp` -> `PrimitiveNode::renderSelf()` (`glColor3f`)

P: Co to jest `GL_MODULATE` przy teksturze?

O: Tryb łączenia tekstury z kolorem/oswietleniem. `MODULATE` mnoży kolor tekstury przez kolor oświetlenia obiektu - dzięki temu oteksturowany obiekt nadal reaguje na światło (ciemnieje w cieniu). Gdyby było `GL_REPLACE`, tekstura zastąpiłaby oświetlenie całkowicie.

Gdzie: `PrimitiveNode.cpp` -> `renderSelf()` (`glTexEnvf ... GL_MODULATE`)

Pojecia: normalne i UV

P: Co to jest normalna i po co ja podajesz?

O: Normalna to wektor prostopadły do powierzchni - mówi, w którą stronę 'patrzy' ścianka. OpenGL używa jej do liczenia oświetlenia: im bardziej normalna skierowana do światła, tym jaśniejszy punkt. Bez normalnych model byłby płaski, bez cieniowania.

Gdzie: `PrimitiveNode.cpp` -> `glNormal3f` w każdym `drawGeometry()`

P: Co to są współrzędne UV?

O: To współrzędne na teksturze (od 0 do 1). Mówią, który fragment obrazka nalepić na dany wierzchołek. `glTexCoord2f(u,v)` przed `glVertex3f` przypisuje punktowi miejsce na teksturze. `u` to pozioma oś tekstury, `v` pionowa.

Gdzie: `PrimitiveNode.cpp` -> `glTexCoord2f` w każdym `drawGeometry()`

P: Czemu wartości normalnej muszą być znormalizowane?

O: Bo OpenGL liczy oświetlenie z cosinusa kąta, który zakłada wektory długości 1. Nieznormalizowana normalna zafałszowałaby jasność. W silniku mamy też włączone `glEnable(GL_NORMALIZE)`, które normalizuje normalne automatycznie po skalowaniu - to ważne, bo skala psuje długość normalnych.

Gdzie: `Engine.cpp` -> `glEnable(GL_NORMALIZE)`; normalne liczone w `PrimitiveNode.cpp`

Geometria - szescian i walec

P: Jak rysujesz szescian? Ile ścian, jakie prymitywy?

O: 6 ścian jako `GL_QUADS` (czworokąty). Każda ściana ma jedną wspólną normalną (np. przód ma $(0,0,1)$) i 4 wierzchołki z UV od $(0,0)$ do $(1,1)$. Wszystkie współrzędne liczę z połowy boku (`halfSize`), więc szescian jest wycentrowany w środku.

Gdzie: `PrimitiveNode.cpp` -> `CubeNode::drawGeometry()`

P: Jak rysujesz boczna powierzchnie walca?

O: Jako `GL_TRIANGLE_STRIP` po obwodzie - dla każdego kąta stawiam dwa wierzchołki (górny i dolny). Normalna jest pozioma, skierowana od osi na zewnątrz ($\cos(\text{kat}), 0, \sin(\text{kat})$). Góra i dół walca to osobne `TRIANGLE_FAN` (dyski). `slices` to liczba podziałów - więcej = gładzy walec.

Gdzie: `PrimitiveNode.cpp` -> `CylinderNode::drawGeometry()`

P: Czemu normalna na boku walca nie zależy od promienia?

O: Bo normalna to tylko KIERUNEK (na zewnątrz od osi), a nie pozycja. Kierunek jest taki sam niezależnie od tego, jak gruby jest walec - dlatego używam czystego $(\cos, 0, \sin)$ bez mnożenia przez promień. Mnożę przez promień tylko pozycje wierzchołków.

Gdzie: `PrimitiveNode.cpp` -> `CylinderNode::drawGeometry()`

Geometria - sfera, stożek, torus

P: Jak parametryzujesz sferę?

O: Dwoma kątami: theta (od bieguna, $0..pi$) dzieli na poziome pasy (stacks), phi (długość, $0..2pi$) na pionowe wycinki (slices). Pozycja punktu to $(\sin(\theta)\cos(\phi), \cos(\theta), \sin(\theta)\sin(\phi))$ razy promień. Rysuje pasami jako TRIANGLE_STRIP.

Gdzie: PrimitiveNode.cpp -> SphereNode::drawGeometry()

P: Dlaczego dla sfery normalna = pozycja wierzchołka (bez promienia)?

O: Bo w sferze kierunek od środka do punktu jest jednocześnie kierunkiem 'na zewnątrz', czyli normalna. Ten sam wektor używam raz jako normalna (czysty, długość 1) i raz jako pozycję (pomnożony przez promień).

Gdzie: PrimitiveNode.cpp -> SphereNode::drawGeometry()

P: Czemu stożek rysujesz osobnymi trójkątami, a nie TRIANGLE_FAN?

O: W wachlarzu czubek to jeden wspólny wierzchołek z jedną normalną - cieniowanie wokół czubka wygląda płasko/brzydko. Rysując N osobnych trójkątów, czubek każdego dostaje normalną uśrednioną z dwóch sąsiednich ścianek, więc światło płynnie przechodzi dookoła stożka.

Gdzie: PrimitiveNode.cpp -> ConeNode::drawGeometry() (pętla GL_TRIANGLES)

P: Jak liczysz nachylenie normalnej na bocznej ścianie stożka?

O: Z proporcji promienia do wysokości. $L = \sqrt{r^2 + h^2}$ to długość tworzącej. Składowa pozioma normalnej to $n_h = h/L$, pionowa $n_v = r/L$. Pionowa jest dodatnia, bo stożek zwęża się ku górze, więc powierzchnia lekko patrzy w górę. Normalna w kierunku kąta: $(\cos n_h, n_v, \sin n_h)$.

Gdzie: PrimitiveNode.cpp -> ConeNode::drawGeometry() (L, n_h, n_v)

P: Jak parametryzowany jest torus?

O: Dwoma kątami u (wokół całego pierścienia, $0..2pi$) i v (wokół rurki, $0..2pi$). Pozycja: $((R+r\cos v)\cos u, r\sin v, (R+r\cos v)\sin u)$, gdzie R to główny promień (do środka rurki), r mały (grubość rurki). Normalna to kierunek od środka rurki do punktu. Torus jest u nas tylko dekoracyjny (zyrandol).

Gdzie: PrimitiveNode.cpp -> TorusNode::drawGeometry()

Plaszczyzna i UV scaling

P: Co robi setUVScale w PlaneNode i jak jest zaimplementowane?

O: Mnoży współrzędne UV. $u_{\text{Max}} = \text{planeWidth} * \text{planeUVScale}$, $v_{\text{Max}} = \text{planeDepth} * \text{planeUVScale}$. Domyślnie skala 1 daje 1 powtórzenie na metr - na ścianie 30m to 30 drobnych kafelków. Mniejsza skala (0.2) daje większe kafelki. Sama geometria się nie zmienia, tylko UV.

Gdzie: PrimitiveNode.cpp -> PlaneNode::drawGeometry(); PlaneNode.h -> setUVScale()

P: Cemu PlaneNode ma normalna w gore (+Y)?

O: Bo to plaszczyzna pozioma (w plaszczyznie XZ), domyslnie podloga. Normalna (0,1,0) patrzy w gore. Zeby zrobic z niej sciane czy sufit, Lukasz obraca ja w Room.cpp - obrot zmienia tez kierunek normalnej.

Gdzie: PrimitiveNode.cpp -> PlaneNode::drawGeometry() (glNormal3f(0,1,0))

Obstacles - rozmieszczenie

P: Ile i jakich przeszkod jest w pokoju?

O: 6 kolumn (CylinderNode), 3 stozki (ConeNode) i 2 skrzynie (CubeNode ze skala). Definiuje je jako listy struktur (CylDef, ConeDef, BoxDef) z pozycja i wymiarami, a potem w petli tworze wezel sceny i wpis kolizji. Rozmieszczone sa po calym pokoju (przednia i tylna polowa).

Gdzie: Obstacles.cpp -> buildObstacles() (wektory cyls, cones, boxes)

P: Cemu skrzynia to CubeNode ze skala, a nie osobna klasa?

O: Bo skrzynia to po prostu przeskalowany szescian. Zamiast pisac nowy ksztalt, tworze CubeNode o boku 1 i ustawiam setScale(w,h,d). Macierz lokalna (Lukasz) zajmie sie skalowaniem. Mniej kodu, ten sam efekt.

Gdzie: Obstacles.cpp -> buildObstacles() (node->setScale przy skrzyniach)

P: Cemu przechowujesz przeszkode dwa razy (wezel sceny + dane kolizji)?

O: Wezel (CylinderNode) sluzi tylko do RYSOWANIA. Do FIZYKI potrzebuje czystych danych (pozycja, promien, wysokosc) bez calego obiektu OpenGL. Rozdzielenie 'co widac' od 'co jest fizyczne' jest czytelniejsze i szybsze - kolizje nie musza znac OpenGL.

Gdzie: Obstacles.cpp -> buildObstacles(); ShootingGallery.h -> cylinderObs_, coneObs_, boxObs_

Obstacles - kolizje gracza

P: Jak dziala wypychanie gracza z kolumny (krok po kroku)?

O: Licze wektor od osi kolumny do gracza w plaszczyznie XZ (dx,dz) i kwadrat odleglosci distSq. Jezeli $\text{distSq} < (\text{promien kolumny} + \text{promien gracza})^2$, gracz jest w srodku. Licze $\text{dist} = \sqrt{\text{distSq}}$, potem $\text{push} = (\text{minR} - \text{dist}) / \text{dist}$ i przesuwam gracza o $\text{dx} * \text{push}$, $\text{dz} * \text{push}$ - czyli wzdluz linii srodek->gracz az na brzeg.

Gdzie: Obstacles.cpp -> applyObstacleCollision() (petla po cylinderObs_)

P: Cemu w kolizjach uzywasz distSq, a nie samej odleglosci?

O: Pierwiastek (sqrt) jest wolny. Porownanie $\text{distSq} < \text{minR} * \text{minR}$ daje ten sam wynik co $\text{dist} < \text{minR}$, ale bez pierwiastka. Sqrt licze dopiero gdy faktycznie musze wypchnac gracza (do policzenia kierunku).

Gdzie: Obstacles.cpp -> applyObstacleCollision()

P: Jak działają kolizje ze skrzynia (prostopadłościanem)?

O: Znajduje punkt na powierzchni skrzyni najbliższy graczowi - przez clamp współrzędnych gracza do zakresu [boxMin, boxMax]. Jeżeli gracz jest bliżej tego punktu niż jego promień, wypycham go na zewnątrz. Gdy gracz wszedł do samego środka ($\text{distSq} \sim 0$), wybieram najbliższą ścianę (min z 4 odległości) i wypycham w jej stronę.

Gdzie: `Obstacles.cpp` -> `applyObstacleCollision()` (petla po `boxObs_`, `clamp + 4` ściany)

P: Czemu stożek w kolizjach traktujesz jak walec?

O: Bo gracz i tak nie wejdzie pod zwężający się nawis stożka (jest za niski/za szeroki u dołu). Wystarczy traktować go jak walec o promieniu podstawy - prostsze, a efekt dla gracza identyczny. Pełny kształt stożka liczę dopiero przy strzale (raycast).

Gdzie: `Obstacles.cpp` -> `applyObstacleCollision()` (petla po `coneObs_`)

Obstacles - blokowanie strzału

P: Jak sprawdzasz, że przeszkoda zasłania cel?

O: Po znalezieniu trafionego celu mam odległość do niego (maxT). W `obstacleBlocksRay` sprawdzam, czy któraś przeszkoda jest przecięta przez ten sam promień przy $t < \text{maxT}$. Jeżeli tak - przeszkoda jest bliżej niż cel, więc strzał w nią trafia pierwszy i liczy się jako pudło.

Gdzie: `Obstacles.cpp` -> `obstacleBlocksRay()`;wołane z `Gameplay.cpp` `onShoot()`

P: Jakich funkcji używasz do sprawdzenia przecięcia z każdym typem przeszkody?

O: `rayCylinderIntersect` dla kolumn, `rayConeIntersect` dla stożków, `rayAABBIntersect` dla skrzyni. Wszystkie trzy napisał Lukasz w `Math3D`. Ja przechodzę petlę po moich listach przeszkód i wołam odpowiednią funkcję, sprawdzając czy t mieści się w $(0, \text{maxT})$.

Gdzie: `Obstacles.cpp` -> `obstacleBlocksRay()`; `Math3D.cpp` (implementacje raycastów)

P: Co by się stało, gdybyś nie sprawdzał blokowania strzału?

O: Gracz mógłby strzelać przez ściany i kolumny - trafiałby cele schowane za przeszkodami, czego nie widzi. Byłoby to nierealistyczne i psułoby sens chowania się celów. Dlatego raycast do celu jest 'przycinany' przez przeszkody.

Gdzie: `Obstacles.cpp` -> `obstacleBlocksRay()`